C++ Concurrency in Action, 2ed.

Brian

May 1, 2026

Contents

1 1. Hello, world of concurrency in C++!

- Concurrency is two or more things happening at the same time
- In programming, this is when two or more things happen in parallel
- Two main reasons to use concurrency:
  1. Seperation of Concerns: Things that pertain to a specific task can be run seperately from the rest of the application
  2. Can run things at the same time, improving performance

2 2. Managing Threads

2.1 Launching Threads

- Every C++ program has at least one thread, which runs `main()`.
- A program can launch other threads that have different functions as entrypoints.
- A thread exits when the function returns, similar to how the program exits when `main()` returns.
- Simple case:

```
void do_some_work();
std::thread my_thread(do_some_work);
```

- `std::thread` works on any callable type, including ctors, where the object gets copied into the thread (invokes copy constructor!)

```
class background_task {
public:
    void operator() () const {
        do_something();
        do_something_else();
    }
};
background_task f;
std::thread my_thread(f);
```

- Prefer uniform initialization when creating a thread, to avoid most vexing parse: if you pass a prvalue to instantiate a thread, this is ambiguous with a function declaration (i.e. `std::thread my_thread(background_task());` is a function declaration),
- To prevent most vexing parse:

```
std::thread my_thread((background_task())); //extra parenthesis
std::thread my_thread{background_task()}; //uniform initialization,
    more idiomatic
```

- To construct a thread, we need a callable; lambda expressions are also commonly used.

```
std::thread my_thread([] {
    do_something();
    do_something_else();
});
```

- Ensure to explicitly state whether to wait for the thread to finish or leave it to run on it's own; if this is not decided before the thread is destroyed, then the program exits (`std::thread` dtor invokes `std::terminate()`)
- Also, it is imperative to ensure that the thread does not access data that it may outlive (important for if you leave the thread to run).
- Same as use-after-free UB, but more tricky to find. Example:

```
struct func {
    int &i;
    func(int& i_): i(i_) {}
    void operator() () {
        for (unsigned j = 0; j < 1000000; ++j) std::cout << i << "\
           n"; //dangling reference
    }
};

void oops() {
    int some_local_state{0};
    func my_func(some_local_state);
    std::thread my_thread(my_func);
    my_thread.detach(); //dont wait for thread to finish
} // local variable some_local_state goes out of scope; thread
   still has reference

int main() {
    oops();
    std::cout << "Here!\n"; //prevent compiler optimizing this away
    return 0;
}
```

- `std::thread::join` will ensure to wait for the associated thread to complete.
- In the above example, replacing `my_thread.detach()` with `my_thread.join()` will make sure that the function exits only after the thread is completed.
- Once `join()` exits, it will clean up all resources associated with the thread, i.e. the `std::thread` object itself is invalid.
- calling `join()` once on a thread will set `joinable()` to be false, i.e. you cannot join the thread anymore.
- Note that a `join()` may be skipped if an exception is thrown after the thread is started but before `join()` is called.
- This can be avoided using a try-catch, however this is very verbose for a pattern that would be used often, and is thus unideal.

```
struct func;
void f() {
    int some_local_state{0};
    func my_func(some_local_state);
    std::thread t(my_func);
    try {
        do_something_in_current_thread();
    } catch {
        t.join();
        throw;
    }
    t.join();
}
```

- It is better to use RAII, and utilize a wrapper type:

```
class thread_guard {
//barebones thread RAII wrapper
    std::thread& t;
public:
    explicit thread_guard(std::thread& t_) : t(t_) {}
    ~thread_guard() {
        if (t.joinable()) t.join();
    }
    thread_guard(thread_guard const&) = delete;
    thread_guard& operator=(thread_guard const&) = delete;
}
```

- Here, `join()` will automatically be called on the thread when it is about to go out of scope.
- However, if the user has explicitly called `join()` already, then it will do nothing.

```
struct func;
void f() {
    int some_local_state{0};
    func my_func(some_local_state);
    std::thread t(my_func);
    thread_guard g(t);
    do_something_in_current_thread();
    // less verbose, and thread_guard is reusable
}
```

- Note that in C++20+, `std::jthread` (joinable thread) was added that basically is a thread with RAII wrapper.
- Calling `detach()` on a thread will leave the thread to run in the background.
- Note that internally, this severes the connection between the actual `std::thread` object and the thread spawned by the os.
- It is thus impossible to communicate with the thread after it gets detached, and you cannot get a `std::thread&` to the detached thread.
- Detached threads are often referred to daemon threads.
- Detached threads are useful for tasks that must run in the background, dont require two way communication, and last for the duration of the program.

- Such tasks include filesystem monitoring, clearing unused entries, optimizing data structures, thread deadlock watchdogs, etc.
- In order to prevent the UB behaviour described above, it is extremely important to ensure that detached threads never access local variables by reference.
- If absolutely needed (for some reason) pass by value, or if it is large then use a `std::shared_ptr`.
- Consider the following example: a word processing application that can support processing multiple documents. This should be ran as a single process, however in order to be able to multiple documents independently each document should have its own thread. Since each dcoument is independent, they should be detached from each other.

```
void edit_document(std::string const& filename) {
    open_document_and_display_gui(filename);
    while (!done_editing()) {
        user_command cmd = get_user_input();
        if (cmd.type == open_new_document) {
            std::string const new_name = get_filename_from_user();
            std::thread t(edit_document,new_name);
            t.detach();
        } else {
            process_user_input(cmd);
        }
    }
}
```

2.2 Passing arguments to a thread function

- As seen in the snippet above, passing arguments to the callable function is done by adding more arguments to the `std::thread` ctor.
- By default, this will create a copy of the arguments for the thread, and then are fowarded as rvalues to the function. In the case above, we must ensure to define the signature of f to accept a `std::string const&`, to ensure that it can bind to rvalue references.
- As well, note that the promotion from a `const char*` to a `std::string` only in the context of the new string. Consider:

```
void f(int i, std::string const& s);
void oops(int some_param) {
    char buffer[1024];
    sprintf(buffer, "%i", some_param);
    std::thread t(f, 3, buffer);
    t.detach();
}
```

- In the snippet above:
  1. On the main thread: `oops()` pushes `buffer` to the stack.
  2. `std::thread` ctor is called,
  3. and it sees `char[]` and decays to `char*`.
  4. The thread `t` receives a copy of the address that `buffer` points to.
  5. `t.detach()` is called, and the `oops()` exits.

6. The stack frame is popped, and `buffer` is freed.
7. The thread now converts `buffer` to a `std::string`, but it contains freed memory.
8. Program crashes due to use-after-free.

- To prevent this, explicitly cast `buffer` to `std::string` when passing to ctor.
- The inverse of this issue can also occur: you want a non-const reference, but the object is copied and passed by value.

```
void update_data_for_widget(widget_id w, widget_data& data);
void oops_again(widget_id w) {
    widget_data data;
    std::thread t(update_data_for_widget, w, data);
    display_status();
    t.join();
    process_widget_data(data);
}
```

- In this case, the program will not compile since `data` is passed by value when it expects a `widget_data&`. Instead of, once again, relying on the implicit conversion, we can explicitly state to pass a reference through `std::thread t(update_data_for_widget, w, std::ref(data))`.
- You can also pass a member function pointer as the function, provided that you supply a suitable a suitable object pointer with it:

```
class X {
public:
    void do_something();
};
X my_x;
std::thread t(&X::do_something(), &my_x);
```

- To use move semantics, and especially in the case of `std::unique_ptr`, `std::move` must explicitly be invoked.

```
void do_something(std::unique_ptr<big_object>);
std::unique_ptr<big_object> p(new big_object);
p->prepare_data(42);
std::thread t(do_something, std::move(p));
```

2.3 Transferring ownership of a thread

- Threads cannot be copied but they can be moved
- Below is an example that creates two threads and transfers ownership amongst three `std::thread` instances.

```
void foo();
void bar();
std::thread t1(foo);
std::thread t2 = std::move(t1);
t1 = std::thread(bar);
std::thread t3;
t3 = std::move(t2);
t1 = std::move(t3); // terminates the program!
```

1. A new thread is started and associated with `t1`.
2. Ownership of the thread is transferred from `t1` to `t2`.
3. `t1` is now associated with a thread of execution, `t2` is associated with the thread running `foo`.
4. A new `std::thread` instance is created `t3`, but not associated with a thread of execution.
5. `t1` is then transferred ownership to a `std::thread` prvalue running `bar`, thus the move is implicit.
6. Ownership of the thread running `foo` is then transferred from `t2` to `t3`. Now, `t1` is associated with the thread running `bar`, `t2` is invalid, and `t3` is associaed with the thread running `foo`.
7. Attempting to move `t3` to `t1` will cause the program to terminate, as `t1` is already associated with an existing thread of execution.

- Since threads can be moved, they can also be returned from functions:

```
std::thread f() {
    void some_function();
    return std::thread(some_function);
}
```

- `std::jthread` RAII implementation (C++20 and newer, uses concepts):

```
#include <concepts>

class joining_thread {
    std::thread t;
public:
    joining_thread() noexcept = default;

    template<typename Callable, typename... Args>
    requires std::invocable<Callable, Args...>
    explicit joining_thread(Callable&& func, Args&&... args) :
        t(std::forward<Callable>(func), std::forward<Args>(args)
            ...) {}

    ~joining_thread() {
        if (joinable()) join();
    }

    joining_thread(const joining_thread&) = delete;
    joining_thread& operator=(const joining_thread&) = delete;

    joining_thread(joining_thread&& other) noexcept :
        t(std::move(other.t)) {}

    joining_thread& operator=(joining_thread&& other) noexcept {
        if (joinable()) join();
        t = std::move(other.t);
        return *this;
    }
};
```

- In the case where threads are used to subdivide work done in an algorithm, you must check that all threads have finished. We can write code like the following which spawns a number of threads and waits for them all to finish.
- Note that this only works with moveable containers.

```
void do_work(unsigned id);
void f(unsigned k) {
    std::vector<std::thread> threads;
    threads.reserve(k);
    for(unsigned i = 0; i < k; ++i) {
        threads.emplace_back(do_work, k); // spawns k threads
    }
    for (auto& t: threads) {
        t.join(); // calls join on each thread, ensuring that they
            are all completed before f() returns
    }
}
```

2.4 Choosing the number of threads at runtime

- The number of truly concurrent threads that are available depends on the machine that is running the program.
- `std::thread::hardware_concurrency()` provides a hint for how many truly concurrent threads are avaiable: note that this information may not be available, and will return `0`.

2.5 Identifying threads

- Thread identifier is of type `std::thread::id`.
- The id for the thread associated with a specific `std::thread` object is retrieved via the `get_id()` member function. This will return the default constructed `std::thread::id` object if it doesnt have an associated thread.
- Calling `std::this_thread::get_id()` which is in the thread header will also retrieve the thread id for the thread in which it was called on.
- thread ids can be comapred, hashed, sorted, etc.

3 Sharing data bewteen threads

3.1 Problems with sharing data bewteen threads

- All issues with sharing data between threads come from modifying the data.

Definition 3.1 (Invariant)**.** A statement that is always true about a particular data stucture. E.g. "This variable contains the number of items in a list".

- To an external observer, invariants must always hold. However, while changing a data structure the state of the invariant may be invalid.
- If a thread were to attempt to access the invariant in this invalid state, it may cause bugs or crashes.

- Consider a doubly linked list. We can say that an invariant for this data structure is that if the "next" pointer for node A points to node B, then the "previous" pointer for node B points to node A.
- While removing a node from this list, we must update two pointers, one on either side.
- The invariant is broken during the time when one pointer is updated but the other is not. Concretely, we outline the steps to delete a node:
  1. Identify node to delete N.
  2. Update link from N-1 to point to N+1.
  3. update the link from N+1 to point to N-1.
  4. delete N.
- During the time between steps 2 and 3, the invariant does not hold. If a thread were to modify data that is of broken invariants, or in this case the "previous" pointer for N-1, we get what is called a *race condition*.

  Definition 3.2 (Race Condition)**.** When the outcome of something depends on the relative ordering of the order of execution of two or more threads; the threads race to complete their respective operation.
- Problematic race conditions can be catastrophic; in the example above, if we were to get the scenario described then we are accessing freed memory.
- According to the C++ spec, data races are considered undefined behaviour.
- Race conditions are difficult to debug as they are not always consistent; if an operation only takes a few cpu instructions and is done consecutively, then the window for a race condition to exist is very small. This causes the bug to not be consistently reproducible.
- There are two main ways to deal with this: *lock-based* (slower, easier to implement and reason about) vs *lock-free* (faster, much more difficult to implement). Lock-free programming and data structures are covered later, as they require deep knowledge and nuance in the C++ memory model.

3.2 Protecting shared data with mutexes

Definition 3.3 (Mutex)**.** A mutex (**Mut**ual **ex**clusion) is a thread synchronization technique that revolves around locking the shared data before accessing it, to ensure no other threads can access the data. Mutexes are implemented in C++ via the `<mutex>` header. To lock it, use `lock()` and `unlock()` to unlock; generally, however, it is not recommended to lock/unlock your code directly, as this requires you to remember to call `unlock()` every time. Instead, C++ provides a RAII wrapper for a mutex via `std::lock_guard`.

```cpp
#include <list>
#include <mutex>
#include <algorithm>

std::list<int> my_list;
std::mutex my_mutex;
void add_to_list(int new_value) {
    std::lock_guard<std::mutex> guard(my_mutex);
    my_list.push_back(new_value);
}
bool list_contains(int value_to_find) {
    std::lock_guard<std::mutex> guard(my_mutex);
    return std::find(my_list.begin(), my_list.end(), value_to_find)
        != my_list.end();
}
```

- In the example above, we have a single global variable `my_list` protected by a single global mutex `my_mutex`. Each function will lock the mutex, ensuring that accesses to each is mutually exclusive: `list_contains()` will never see the list partway through a modificatin by `add_to_list()`.
- Structuring protected data with mutexes also poses an interesting challenge: idiomatically, we would want to package the entire program above into a single class, and abstract away the mutex; interface the data with `add_to_list` and `list_contains`,and have `my_list` be private.
- The issue comes when the member functions return references to the protected data; Now this makes it so that the mutexes are essentially useless.
- As well, you must also ensure that they dont pass references of protected data into functions outside of the mutex's control. For example,

```cpp
class some_data {
    int a;
    std::string b;
public:
    void foo();
};
class wrapper {
    some_data data;
    std::mutex m;
public:
    template<typename Function>
    void process_data(Function func) {
        std::lock_guard<std::mutex> l(m);
        func(data);
    }
};
some_data* unprotected;
void malicious_function(some_data& protected_data) {
    unprotected = &unprotected_data;
}
wrapper x;
void bar() {
    x.process_data(malicious_function);
    unprotected->foo();
}
```

- In the snippet above, the call to the user specifie `func` function means that `bar` can pass in `malicious_function` to bypass the mutex and then call `foo` without the mutex being locked.
- As well, many interfaces for data structures may inherentely contain data races that canont be protected by mutexes. consider the following implementation of a stack:

```
template<typename T, typename Container = std::deque<T>>
class stack {
public:
    explicit stack(const Container&);
    explicit stack(Container&& = Container());
    template <class Alloc> explicit stack(const Alloc&);
    template <class Alloc> stack(const Container&, const Alloc&);
    template <class Alloc> stack(Container&&, const Alloc&);
    template <class Alloc> stack(stack&&, const Alloc&);
    bool empty() const;
    size_t size() const;
    T& top();
    T const& top() const;
    void push(T const&);
    void push(T&&);
    void pop();
    void swap(stack&&);
    template <class... Args> void emplace(Args&&... args)
};
```

- The issue with this is that the functions `empty()` and `size` cannot be trusted. Consider:

```
1 stack<int> s;
2 if(!s.empty()) { //Another thread calls pop(), race condition!
3     int const value = top();
4     s.pop();
5     do_something(value);
6 }
```

- As well, there exists another race condition, in between lines 4 and 5. Even if the stack data structure is protected by a lock, we could still get the following case:

Thread A	Thread B
`if(!s.empty())`	
	`if(!s.empty())`
`int const value = s.top();`	
	`int const value = s.top();`
`s.pop();`	
`do_something(value);`	
	`s.pop();`
	`do_something(value);`

- In this case, both thread A and B would read the exact same value for `s.top()`. Intuitively, we would think to wrap the entire block in a lock guard. However, there are issues with data integrity in the case where the objects in the stack raise exceptions. Consider:

```
T foo(stack& data) {
    std::lock_guard<std::mutex> lock(m);
    T result = data.top();
    data.pop();
    return result;
}
```

- Line 3 will invoke the copy constructor for type `T`. This element is then popped off the stack, and then returned to the caller. If an exception were to be raised in lines 3 (during copy construction) or during returning the value, the caller would receive an exception.
- Furthermore, since the data has already been popped, it is permanently lost.
- There are a few ways to amend this, each with their own tradeoffs:
    1. Pass by reference: If we define pop to take a reference of the variable we want to store the popped element to, we cannot end up in the invalid state: if an exception is raised on the copy constuctor, it terminates and the data is not popped. if it fails on the pop, then result already has the popped element. The disadvantage of this implementation is that we must construct an instance of the stack's value prior to calling pop, which may be expensive depending on the type.
    2. Require a noexcept copy constructor or move constructor: Since this only occurs when the copy constructor or move constructors raise an exception, if we were to enforce that the object is has a `noexcept` copy and move constructor then it would circumnavigate this error altogether. This can be in some cases, obviously, not ideal, as we limit the types that our stack supports to only objects that do not throw on copy/move.
    3. Return a shared pointer to the popped item: This is generally seen as the most idiomatic solution to this problem. Since moving a `shared_ptr` is guaranteed to never throw, it is completely safe; it either fully errors out or it fully completes. Note that `shared_ptr` is generally used for this situation as it avoids memory leaks and the standard library implements behaviour for memory allocation that doesn't use `new` and `delete`. This is important because requiring each object on the stack to be allocated via `new` would impose significant overhead.
- Ideally, implementing all three would be ideal:

```
1  struct empty_stack : std::exception {
2      const char* what() const noexcept;
3  };
4  template <typename T>
5  class threadsafe_stack {
6  public:
7      threadsafe_stack();
8      threadsafe_stack(const threadsafe_stack&);
9      threadsafe_stack& operator=(const threadsafe_stack&) = delete;
10     void push(T new_value);
11     std::shared_ptr<T> pop();
12     void pop(T& value);
13     bool empty() const;
14 };
```

- It is very tricky to get the granularity of locking correct; lock too small and the protection doesn't account for all race conditions. Lock too much and you lose benefits of threading altogether.
- The first versions of the linux kernel used a single global lock, meaning that a two processor system had much worse performance than two single-processor systems.

Definition 3.4 (Deadlock)**.** When a pair of threads needs to lock a pair of mutexes, and each thread has one mutex and is waiting for the other. Neither thread can proceed, since each thread is waiting on the other.

- Generally, ensure to lock mutexes in the same order, i.e. always lock mutex A before mutex B.
- `std::lock` is a function that can lock two or more mutexes while guaranteeing that they wont deadlock.

```
class some_big_object;
void swap(some_big_object& lhs, some_big_object& rhs);
class X {
private:
    some_big_object some_detail;
    std::mutex m;
public:
    X(some_big_object const& sd) : some_detail(sd) {}
    friend void swap(X& lhs, X& rhs) {
        if (&lhs == &rhs) return ;
        std::lock(lhs.m, rhs.m);
        std::lock_guard<std::mutex> lock_a(lhs.m, std::adopt_lock);
        std::lock_guard<std::mutex> lock_b(rhs.m, std::adopt_lock);
        swap(lhs.some_detail, rhs.some_detail);
    }
};
```

- Consider the case without `std::lock`:
  1. Thread A calls `swap(X, Y)`. It locks `X.m`, and is about to lock `Y.m`.
  2. At the same time, thread B calls `swap(Y, X)`. It locks `Y.m`, and is about to lock `X.m`.
  3. Both threads are waiting on the other, thus a deadlock occurs.
- `std::lock` handles the logic in the order of locking, to ensure that it always happens in the same order. Since the mutexes are already locked, we pass `std::adopt_lock` to the lock guard to indicate a transfer in ownership of the lock and that the mutex is already locked.
- Since the mutexes are always locked in the same order, there is never a deadlock.
- We can also use `std::scoped_lock<>`:

```
// ...
    std::scoped_lock guard(lhs.m, rhs.m);
    swap(lhs.some_detail rhs.some_detail);
```

- `std::scoped_lock` uses the same algorithm as `std::lock`, and thus the mutexes are always locked in the same order. *note: `std::scoped_lock` requires a variadic template, however since C++17 CTAD allows this to be implicit.*
- Ways to avoid deadlocks:
  1. **Don't wait for another thread if there is any possibility it's waiting for you.**
  2. Avoid nested locks: Don't acquire a lock if you already hold one.
  3. Always acquire locks in a fixed order: try to utilize `std::lock` whenever possible. If this is not possible, then define some fixed order to acquire the locks. For example, in the linked list when deleting a node, define a direction to acquire nodes; if traversing from head->tail, acquire normally, but if traversing from tail->head, then acquire in inverse order. This is known as "hand-over-hand" locking.

- we can also use `std::unique_lock` as well; this is templated with a mutex, and allows a lock to be deferred; i.e. you can define a unique lock and keep it unlocked unil it is needed. As well, it is moveable, and thus ownership can be transferred and it can be returned from functions.

```
// ...
    std::unique_lock<std::mutex> lock_a(lhs.m, std::defer_lock); // keep mutex unlocked
    std::unique_lock<std::mutex> lock_b(rhs.m, std::defer_lock);
    std::lock(lock_a, lock_b); // lock in a safe order
    swap(lhs.some_detail, rhs.some_detail);
```

- The unique lock also contains a flag for if it owns the current lock. If the lock instance owns the mutex, it must unlock the mutex when the dtor is called, else it must not unlock the mutex.
- Generally `scoped_lock` is preferred, unless in times where ownership of a lock needs to be transferred, for example when you want to prepare data safely before returning the lock to the caller:

```
std::unique_lock<std::mutex> get_lock() {
    extern std::mutex some_mutex;
    std::unique_lock<std::mutex> lk(some_mutex);
    prepare_data();
    return lk; // lock is moved, thus dtor does NOT invoke unlock()
}
void process_data() {
    std::unique_lock<std::mutex> lk(get_lock());
    do_something();
}
```

3.3 Alternative facilities for protecting shared data

- Consider the case where an object only needs protection when being constructed, but is then read-only, requiring no explicit thread synchronization.
- Consider the following example for single threaded lazy evaluation:

```
std::shared_ptr<some_big_resource> resource_ptr;
void foo() {
    if (!resource_ptr) {// only construct some_big_resource if it has not already been constructed
        resource_ptr.reset(new some_big_resource);
    }
    resource_ptr->do_something();
}
```

- if `some_big_resource` were read-only, we would only need to protect the initialization. However, wrapping this in a mutex would cause threads to always lock this portion, even if it has already been constructed:

```
std::shared_ptr<some_big_resource> resource_ptr;
std::mutex resource_mutex;
void foo() {
    std::unique_lock<std::mutex> lk(resource_mutex); // A lot of contention here!
    if (!resource_ptr) {
```

```
        resource_ptr.reset(new some_big_resource);
    }
    lk.unlock();
    resource_ptr->do_something();
}
```

- Fix: use the `std::once_flag` and `std::call_once`.
- `std::once_flag` tracks three states: not started, in progress, and completed.
- If a thread tries to access the critical section while the first one is still working, it gets blocked. If the critical section throws an exception, the flag is reset to not started.
- In C++11+, a static local variable can be used as well, giving identical behaviour.

```
std::once_flag resource_flag;
void init_resource() {
    resource_ptr.reset(new some_big_resource);
}
void foo() {
    std::call_once(resource_flag, init_resource);
    resource_ptr->do_something();
}
// OR
std::shared_ptr<some_big_resource> get_resource() {
    std::shared_ptr<some_big_resource> res = std::make_shared<
        some_big_resource>(); //Guaranteed to be thread safe, C++11+
    return res;
}
void foo() {
    get_resource()->do_something();
}
```

- *Note that the ctor fro `some_big_resource` must not invoke `foo()`, as this would cause a circular dependency and deadlock.*
- Shared mutex: Useful for when you have multiple readers but only one writer. A shared mutex allows multiple readers at the same time, but they get blocked when the writer is accessing the data.

```
#include <shared_mutex>
std::shared_mutex sm;
std::map<int, int> data;
// Multiple readers
void read_data(int key) {
    std::shared_lock<std::shared_mutex> lock(sm);
    auto val = data[key];
}
// Single writer
void write_data(int key, int val) {
    std::lock_guard<std::shared_mutex> lock(sm); //or std::
        unique_lock
    data[key] = val; //all readers are blocked
}
```

4 Synchronizing concurrent operations

5 The C++ memory model and operations on atomic types

5.1 Memory model basics

- Two main aspects to the memory model: the basic structural aspects, which pertain to how things are layed out in memory, and the concurrent aspects.
- All data in C++ is made up of *objects*.
- An object, in this case, is defined as "a region of storage".
- An object is stored in one or more memory locations. Each memory location is either an object of a scalar type, such as integral types, or a sequence of adjacent bit fields.
- While bit fields themselves are objects, they are still counted to be at the same memory location. Consider:

```
1  struct my_data {
2      int i;
3      double d;
4      unsigned bf1:10;
5      int bf2:25;
6      int :0;
7      int bf4:9;
8      int i2;
9      char c1, c2;
10     std::string s;
11 };
```

- Would look something like this in memory:

i
d
bf1 | bf2
bf4
i2
c1
c2
s (multiple contiguous locations)

- the `int :0` is an anonymous zero size bitfield, signalling the compiler to seperate `b1`, `b2` with `b4`.
- Important: Every variable is an object, including those that are members of other objects.
- Every object occupies at least one memory location.
- Variables of primative types occupy exactly one memory location.
- Adjacent bitfields are part of the same memory location.
- In C++, two different threads can safely modify the data within two different memory locations at the same time.
- In the example above, it would not be safe to update `bf1` and `bf2` at the same time, as they occupy the same memory location.

- If there is no enforced ordering between two accesses to the same memory location, this is a data race which is defined to be undefined behaviour.
- Any instance of undefined behaviour in a program means that the entire program is undefined, and it may do anything.
- This undefined behaviour can be avoided via atomic operations, even if there exists a data race.
- Every C++ object has a modification order, which describes all the writes to that object from all threads, starting with intialization.
- Most cases this varies between runs, however within a given run all threads must agree on the order.

5.2 Atomic operations and types in C++

Definition 5.1 (Atomic operation)**.** An operation that is indivisible. There does not exist a "half-done" state; it is either done or not done. The C++ compiler guarantees that if a read to a value of an object is atomic, then all modifications done to that object will also be atomic; This, however, does not guarantee that the read will be of the latest state of the object.

- Atomic types exist in the `<atomic>` header.
- Depending on the context, a `std::atomic<type>` may just be the type wrapped in a mutex. This can be checked via `std::atomic<type>::is_lock_free()`.
- Example: a `std::atomic<long long>` may only be atomic if it is aligned to an 8-byte boundary in memory. If it is not, the compiler may fall back to a mutex.
- Since C++17, all atomic types have a `static constexpr` member variable, `X::is_always_lock_free`. This holds the value 0 if `X` is never lock free, 2 if it is always lock free in all contexts, and 1 if it depends on the use case.
- The only atomic type that doesnt have a `is_always_lock_free` is the `std::atomic_flag`, which is an atomic boolean type. This is required to always be lock free, no matter the hardware. Note that this is distinct from `std::atomic<bool>`.
- The `std::atomic_flag` has two member functions: `test_and_set()`, which queries the flag and sets it, and `clear()` which resets it.
- Most atomic types have no copy constructor or assignment.
- Most atomic types have `load()`, `store()`, `exchange()`, `compare_exchange_weak()`, and `compare_exchange_strong()`.
- They also support compound assignment operators when applicable: `+=`, `-=`, etc.
- The compound operators also have named functions: `fetch_add()`, `fetch_sub()`, etc.
- Using the compound operator itself will return the value after the change; using the named function will return the value before the change.
- All atomic operations also have an optional memory-ordering argument. This is one of the values in the `std::memory_order` enum, which can be one of the following:
  1. `std::memory_order_relaxed`
  2. `std::memory_order_acquire`
  3. `std::memory_order_consume`

4. `std::memory_order_acq_rel`
5. `std::memory_order_release`
6. `std::memory_order_seq_cst`

- If this is not specified, the default argument is `std::memory_order_seq_cst`, the strongest memory ordering.
- The orderings are seperated into three categories, depending on the operation being done:
  1. *Store* operations, which can have `memory_order_relaxed` `memory_order_release`, or `memory_order_seq_cst`
  2. *Load* operations, which can have `memory_order_relaxed`, `memory_order_consume`, `memory_order_acquire`, or `memory_order_seq_cst`
  3. *Read-Modify-Write* operations, which can have any memory ordering.

5.2.1 Operations on `std::atomic_flag`

- `std::atomic_flag` is the simplest atomic type. It is deliberately very basic, and isn't meant to be used directly.
- Rather, `std::atomic_flag` is to be used as a buiding block for other atomic types.
- `std::atomic_flag` must be initialized with the `ATOMIC_FLAG_INIT`.

```
1 std::atomic_flag f = ATOMIC_FLAG_INIT;
```

- There are three things you can do with an atomic flag: destroy it, clear it, or query it's previous value. These correspond to the dtor, `clear()`, and `test_and_set()` member functions.
- The two member functions can have their memory ordering specified; `clear()` is a store operation and thus cannot have `memory_order_acquire` or `memory_order_acq_rel` semantics, but `test_and_set()` is a RMW thus it can have any ordering.

```
1 // ...
2 f.clear(std::memory_order_release);
3 bool x = f.test_and_set();
```

- While limited, this type can be used to implement a basic spinlock mutex:

```
1  class spinlock_mutex {
2      std::atomic_flag flag;
3  public:
4      spinlock_mutex() : flag(ATOMIC_FLAG_INIT) {}
5      void lock() {
6          while(flag.test_and_set(std::memory_order_acquire));
7      }
8      void unlock() {
9          flag.clear(std::memory_order_release);
10     }
11 };
```

- If the condition in the while loop evaluates to false, this means that the mutex was unlocked. The thread that read this value will then lock the mutex, causing all other threads to spin.

5.2.2 Operations on `std::atomic<bool>`

- `std::atomic<bool>` can be constructed from and assigned to by non-atomic `bool`.

```
std::atomic<bool> b{true};
b = false;
```

- The assignment operator, similar to other atomic types, will return the value instead of a reference.
- This is because returning a reference to an atomic is not necessarily atomic; the value may have changed. Thus, returning the value of the atomic is preferred.
- Writes are done via `store()`, and `test_and_set` is replaced with `exchange()`, allowing the user to set the value to whatever they want.
- There is also support for two compare-exchange operations. This will compare the value of an atomic variable with a supplied expected value, and stores a supplied desired value if they are equal (the operation succeeds, returns true). If they are not equal, then the expected value is updated to be the value previously stored (the operation fails, returns false).
- For `compare_exchange_weak()`, the store may not be successful even if the expected equals the previous. In this case, the value stays unchanged, and the function returns false. This is mainly due to the processor not supporting a single compare-and-exchange instruction, and thus cannot guarantee that it happens atomically. This is known as a *spurious failure*.
- Due to this, `compare_exchange_weak()` is usually used in a loop:

```
bool expected = false;
extern std::atomic<bool> b; //set somewhere else
while (!b.compare_exchange_weak(expected, true) && !expected);
```

- Here, the thread will spin until the CAS succeeds; while expected is false, it fails spuriously.
- On the other hand, `compare_exchange_strong()` is guaranteed to return false iff the value wasn't equal to the expected value. However, on some processors, this may be slower.
- If you want to change the stored value regardless of the expected, then set expected to stored in a loop.
- Compare-exchange takes two memory ordering parameters: first for success, second for fail
- Failed compare-exchange has no write thus memory ordering on this cannot be `memory_order_release` nor `memory_order_acq_rel`.
- Only specifying success ordering wil implicitly set the failure ordering to the same, except for release (which will default to relaxed) and acquire-release (which will default to acquire).

5.2.3 Operations on `std::atomic<T*>`: pointer arthimetic

- has the same basic operations as `std::atomic<bool>`
- Pointer arithmetic is done via `fetch_add` or `fetch_sub`, compound assignment, or with post/pre increment/decrement.
- Note: `+=` and `fetch_add` are not equal! `fetch_add` returns the original, compound assingment returns the modified.

5.2.4 The `std::atomic<>` primary class template

- Any user defined type is exposed the same interface as `std::atomic<bool>`, so long as it meets the following:
  1. Must have a trivial copy-assignment.
  2. By implication, it must also not have any virtual functions or virtual base classes.
  3. Every base class and non-static data member of the type must also have a trivial copy-assignment operator.
- As well, compare-exchange operations do a bitwise comparison via `memcmp`, ignoring any overloaded comparison operators. This may cause a compare-exchange to fail due to things such as padding bits.
- These restrictions are put in place as the compiler treats atomics as a raw block of bits; non-trivial assignment may cause operations to leak references to non-atomic operations, as well as having unpredictable memory layouts due to the added overhead of a virtual pointer/table.
- Atomic types should only be used for simple operations; while atomic operations on more complex types such as `std::vector<>` is possible, it is preferred to just use a `std::mutex`.

5.3 Synchronizing operations and enforcing ordering

- Suppose you have one writer thread that is populating a datastructure to be read by a second thread. In order to prevent any data races, the reader thread may only access the data when a "ready" flag is set by the writer:

```cpp
#include <vector>
#include <atomic>
#include <iostream>
#include <immintrin.h> // obviously were on x86 intel
std::vector<int> data;
std::atomic<bool> data_ready{false};
void reader_thread() {
    while(!data_ready.load()) {
        _mm_pause();
    }
    std::cout << "The answer=" << data[0] << "\n";
}

void writer_thread() {
    data.push_back(42);
    data_ready = true;
}
```

- Without memory ordering, this would be UB as we have concurrent reading/writing to a non-atomic type without an enforced ordering.
- The flag `data_read` enables the memory model relations **happens-before** and **synchronizes-with**:
  - The write of the data happens before the write to the `data_ready` flag.
  - The read of the flag happens before the read of the data.
  - By making the reader spin up until the data is ready, we synchronize the reader-writer threads.

Definition 5.2 (Synchronizes-with)**.** Consider a suitably-tagged atomic write operation `W` on an atomic `x`. This synchronizes with a suitably-tagged atomic read operation if:

1. it reads the value on `x` that `W` wrote,
2. the same thread that wrote `W` does some number of atomic writes onto `x` and is then read (in which case it would synchronize with the original operation `W`),
3. or there is a sequence of atomic RMW operations performed by any thread on `x` where each RMW reads the value written by the previous (similarly, it would synchronize with the original operation `W`).

- Much of the nuance in the above definition comes from "suitably-tagged", as the C++ memory model allows you to define how strictly to enforce memory ordering.

Definition 5.3 (Happens-before)**.** Specifies which operations see the effects of other operations. An operation A happens before B iff:

1. **Sequenced-before**: A and B are on the same thread, and A comes before B in code.
2. **Synchronizes-with**: A release-stores in thread 1, and B acquire-loads in thread 2 the value that A wrote.
3. **Transitivity**: if A happens-before X, and X happens-before B, then A happens-before B.

In general, if two operations occur in the same statement, there is no happens-before relationship between them and they are unordered.

- C++ has 6 memory ordering options, however they correspond to three models: sequentially consistent ordering, acquire-release ordering, and relaxed ordering.

5.3.1 Sequentially consistent ordering

- Default ordering: implies that the behaviour of the program is consistent with a sequetial view.
- Behaviour of a sequentially consistent multithreaded program is the exact same as if it were to be ran on a single thread.
- In terms of synchronization, sequentially consistent ordering enforces a two-way ordering: Consider a store that is synchronzied with a load on a different thread. Any sequentially consistent operation done after the load must also appear after the store.
- On more weakly-ordered architectures (such as ARM), sequentially consistent ordering may impose a noticable performance penalty.

```cpp
#include <atomic>
#include <thread>
#include <assert.h>
std::atomic<bool> x, y;
std::atomic<int> z;
void write_x() {
    x.store(true, std::memory_order_seq_cst); //t_1 // equivalent
       to x.store(true);
}
void write_y() {
    y.store(true, std::memory_order_seq_cst); // t_2
}
void read_x_then_y() {
    while (!x.load(std::memory_order_seq_cst));
    if (y.load(std::memory_order_seq_cst)) ++z; // t_3
}
void read_y_then_x() {
    while (!y.load(std::memory_order_seq_cst));
    if (x.load(std::memory_order_seq_cst)) ++z; // t_4
}
int main() {
    x = false;
    y = false;
    z = 0;
    std::thread a{write_x};
    std::thread b{write_y};
    std::thread c{read_x_then_y};
    std::thread d{read_y_then_x};
    a.join();
    b.join();
    c.join();
    d.join();
    assert(z.load() >= 1);// this will never fail
}
```

- In this case, we have that the writes to x and y must appear (in some order) before any reads to x and y appear. Note that the specific ordering of the reads/writes themselves is not enforced. Proof:

Assume $z = 0$. This means that neither thread C nor D incremented z, thus:

$$\text{Thread C saw } y == \text{false at } t_3,$$
$$\text{Thread D saw } x == \text{false at } t_4.$$

Since thread C exited the spin loop, it observed $x ==$ true. By sequentially consistent ordering,

$$t_1 \prec t_3,$$
$$t_2 \prec t_4$$

By definition, sequentially consistent ordering guarantees a strict total order $\prec$ over all atomics. thus, the stores at t_1 and t_2 must be totally ordered, so either:
Case 1: $t_1 \prec t_2$ Then, since $t_2 \prec t_4$,

$$t_1 \prec t_2 \prec t_4 \text{ by transitivity.}$$

Since at t_4 both t_1 and t_2 occured, thread D could not have failed the if check, which is a contradiction.
Case 2: $t_2 \prec t_1$ Then, since $t_1 \prec t_3$,

$$t_2 \prec t_1 \prec t_3 \text{ by transitivity.}$$

Since at t_3 both t_1 and t_2 occured, thread C could not have failed the if check, which is a contradiction. Since both yield a contradiction, $z = 0$ must be false.

$$\therefore z \neq 0.$$

5.3.2 Relaxed ordering

- Operations on atomic tyes with relaxed ordering do not particiapte in synchronizes-with relationships. Operations on the same variable within one thread will still respect happens-before, but there is almost no requirements on ordering relative to other threads.

```
#include <atomic>
#include <thread>
#include <assert.h>
std::atomic<bool> x, y;
std::atomic<int> z;
void write_x_then_y() {
    x.store(true, std::memory_order_relaxed);
    y.store(true, std::memory_order_relaxed);
}
void read_y_then_x() {
    while (!y.load(std::memory_order_relaxed));
    if (x.load(std::memory_order_relaxed)) ++z;
}
int main() {
    x = false;
    y = false;
    z = 0;
    std::thread a{write_x_then_y};
    std::thread b{read_y_then_x};
    assert(z.load() >= 1);
}
```

- This assertion may or may not fire, as the ordering: y stores true, b sees y = true, b sees x = false, x stores true.